Part 1.1 - Load and explore the taxi data

Student Reasoning — Taxi data exploration

What is the shape of the data?

- The shape of the data is right skewed histogram with the highest peak between 0 to 20.

Are there missing values or impossible values (e.g. negative tips, zero-distance trips)?

-There are no missing values because all the columns give 0 as missing values (zero-distance trips).

What does the tip_amount distribution look like (skew, outliers, many zeros)?

-It looks right-skewed which means most of the tips are small but there are very large tip amount stretching to the right. Outliers there very peak tips amounts between 0 to 20.

How will this influence your preprocessing?

-This means that I would need to handle the skew and outliers before training my regression model: Predicting taxi tip_amount (NYC Yellow Taxi).

How to create a feature engineering in supervised learning of a regression model

Use one-hot encoding:

df = pd.get_dummies(df, columns=["City"], drop_first=True)

df = pd.get_dummies(df, columns=["City"], drop_first=True)

Scale Numerical Features

Many regression algorithms perform better when numerical features are on similar scales.

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_scaled = scaler.fit_transform(df[["House_Size", "Age"]])

```
from sklearn.feature_selection import SelectKBest, f_regression

selector = SelectKBest(score_func=f_regression, k=5)
```

Part 1.2 — Preprocessing & feature engineering

Student Reasoning — Taxi preprocessing

1. How did you handle missing/invalid rows and why?
   I drop rows using drop_first = true, by removing distance with less than or equal to zero and keep only non-negative tips.
2. Which new feature(s) did you engineer and what is the intuition behind them?
   I engineer fare_per_mile and total_surcharges. Which will show me the cost of each mile.
3. Which scaling method did you choose and why is it appropriate here?
   I used standardScaler. Its appropriate here because it is the most commonly used to scale numeric features.

Part 1.3 — Train / Validation / Test split

Student Reasoning — Splitting

1. What ratio did you use?

I used a 60/20/20 split (60% training, 20% validation, 20% test). This will help me get enough data for training while keeping a separate validation set

2. Why is a separate validation set useful in addition to a test set?

A separate validation set is useful in addition to a test set because it allows me to compare models, and how well the model can perform with hidden data.

3. Why must the scaler (and any imputation statistics) be fit on the training data only?

The scaler must fit only on the training data to prevent data leakage, so that the validation set remain unseen.

Part 1.4 — Train a regressor and check for overfitting

Student Reasoning — Regression evaluation & overfitting

1. Which model performed best on the validation set, and which hyper-parameters did you try?
   Random forest performed best on the validation set because it has high R^2 but low RMSE compared to linear regression. I use the default hyper-parameters.

2. Compare TRAIN vs VALIDATION vs TEST metrics. Is the model overfitting, underfitting, or well-fitted?Quote the specific numbers that back up your claim.

In Random Forest

Train: RMSE=0.1298, R^2=0.9956

Validation: RMSE=0.3054, R^2=0.9756

Test: RMSE=0.2994, R^2=0.9764

The gap between Train, validation, Test (0.9956 vs 0.9756 vs 0.9764) is small, this means that the model is well-fitted.

In linear regression

Train: RMSE=1.6950, R^2=0.2456

Validation: RMSE=1.7093, R^2=0.2351

Test: RMSE=1.7349, R^2=0.2076

The metrics are low; this means that it is underfitting.

3. What would you change to reduce overfitting if you saw it?

For Random Forest, I would reduce max depth and add minimal split to reduce that small amount of overfitting. For linear regression, I would add more features to the model.

Section 2 — Supervised Learning: Multi-class Classification

Predicting obesity level (NObeyesdad)

Part 2.1 — Load and explore the obesity data

Student Reasoning — Obesity data exploration

How many rows/features are there?

There 2111 rows and 17 features.

Which columns are categorical vs numeric? Is the target balanced across the 7 classes, and why does class (im)balance matter for classification?

8 columns are categorical (gender, overweight, CAEC, SMOKE,SCC,CALC,MTRANS, Nobeyesdad) and 8 columns are numerical, 1 column is empty

(

Age	Height	Weight	FCVC	NCP	CH2O	FAF	TUE

) The target is not balanced because some classes have common inputs. Class imbalance matters because a model may guess the common class and consider it accurate.

Part 2.2 — Preprocessing & feature engineering

Student Reasoning — Obesity preprocessing 1. How did you encode each type of categorical variable, and why?

- Binary yes/no columns (family_history_with_overweight, FAVC, SMOKE, SCC) -> 0/1

-Ordinal-ish columns (CAEC, CALC) and nominal (Gender, MTRANS)

2. Did you engineer any feature (e.g. BMI)? Argue whether it is fair to include given the target is an obesity level.

Yes, I engineer feature BMI: obesity["BMI"] = obesity["Weight"] / (obesity["Height"] ** 2)

3. Which scaler did you use and why?

I use standardscaler to put all features on the same scale.

Part 2.3 — Stratified Train / Validation / Test split

Student Reasoning — Splitting

What split ratio did you choose?

I use split ratio 60/20/20

Why is stratify=y important for this dataset?

stratify=y is important for this dataset, because it keeps the same proportions in each split.

What could go wrong if you split without stratifying?

Without stratifying the model will just guess random split and assign them to classes.

Part 2.4 — Train a classifier and check for overfitting

Student Reasoning — Classification evaluation & overfitting

1. Which classifier did you choose and why?
   I use random forest classifier because knows how to handle non-linear relationships.
2. Compare TRAIN vs VALIDATION vs TEST accuracy/F1. Is the model overfitting, underfitting, or well-fitted? Cite the specific metrics.

   In random forest it gives accuracy of Train, Validation and Test:
   Train - Accuracy: 1.0000
   Validation - Accuracy: 0.9834
   Test - Accuracy: 0.9905
   The accuracy gap between them is very small (1.0000 vs 0.9834 vs 0.9905) the model is well fitted with small overfitting.

3. From the confusion matrix, which obesity levels are hardest to tell apart, and why might that be?
   Overweight level I and II are hardest to tell apart, because they are close to each other.

Section 3 — Unsupervised Learning: K-Means Clustering

Discovering hidden groups in the obesity data

Part 3.1 — Choose k, fit K-Means, and visualise

Student Reasoning — Clustering

1. How did you choose k? Quote the Elbow/silhouette evidence.
   I chose k=6 clusters because at k=6, that's where the elbow bend and that's where the silhouette has the highest peak.
2. Looking at the crosstab, do the unsupervised clusters resemble the real obesity levels? Where do they agree and where do they break down?
   The clusters do not fully resemble the real obesity levels, they do agree at
   Cluster 1: Insufficient and normal weights
   Cluster 2: Very Obese

   And they break at cluster 3: Obesity types and cluster 4 : overweight

3. In a real public-health setting where labels are expensive to collect, what would these clusters be useful for?

In real public health settings these clusters are useful because it helps find the different patient groups of obesity without labels.

Section 4 — Reflection

Answer in a few sentences each:

1. Supervised vs unsupervised: What did the classifier learn that K-Means could not, and vice-versa?

The classifier learned how to predict how to predict obesity without labels using BMI = weight/height. K-Means learned how to group similar people together without labels too. This means the classifier is for prediction, and K-means is for finding hidden patterns.

2. Regression vs classification: How did evaluating a continuous target (tips) differ from evaluating a categorical one (obesity level)?

   For regression (tips) , I used RMSE and R squared to check how far or different the prediction was from actual values.

   For classification of (abesity level) I used accuracy and F1-score to check how many predictions were correct.

3. Overfitting: Across all three tasks, where did you see the biggest train-vs-test gap, and what is the single most effective thing you did (or would do) to close it?

   Across all the three tasks, the biggest train vs test gap is in obesity classification with (Train - Accuracy: 1.0000, Test - Accuracy: 0.9905) and in taxi regression with ((Train R^2: 0.9956 and Test R^2: 0.9756 vs 0.9764) this are small gap of overfitting amount, the single most effective thing I would do is to maximise the max depth limit in Random Forest.